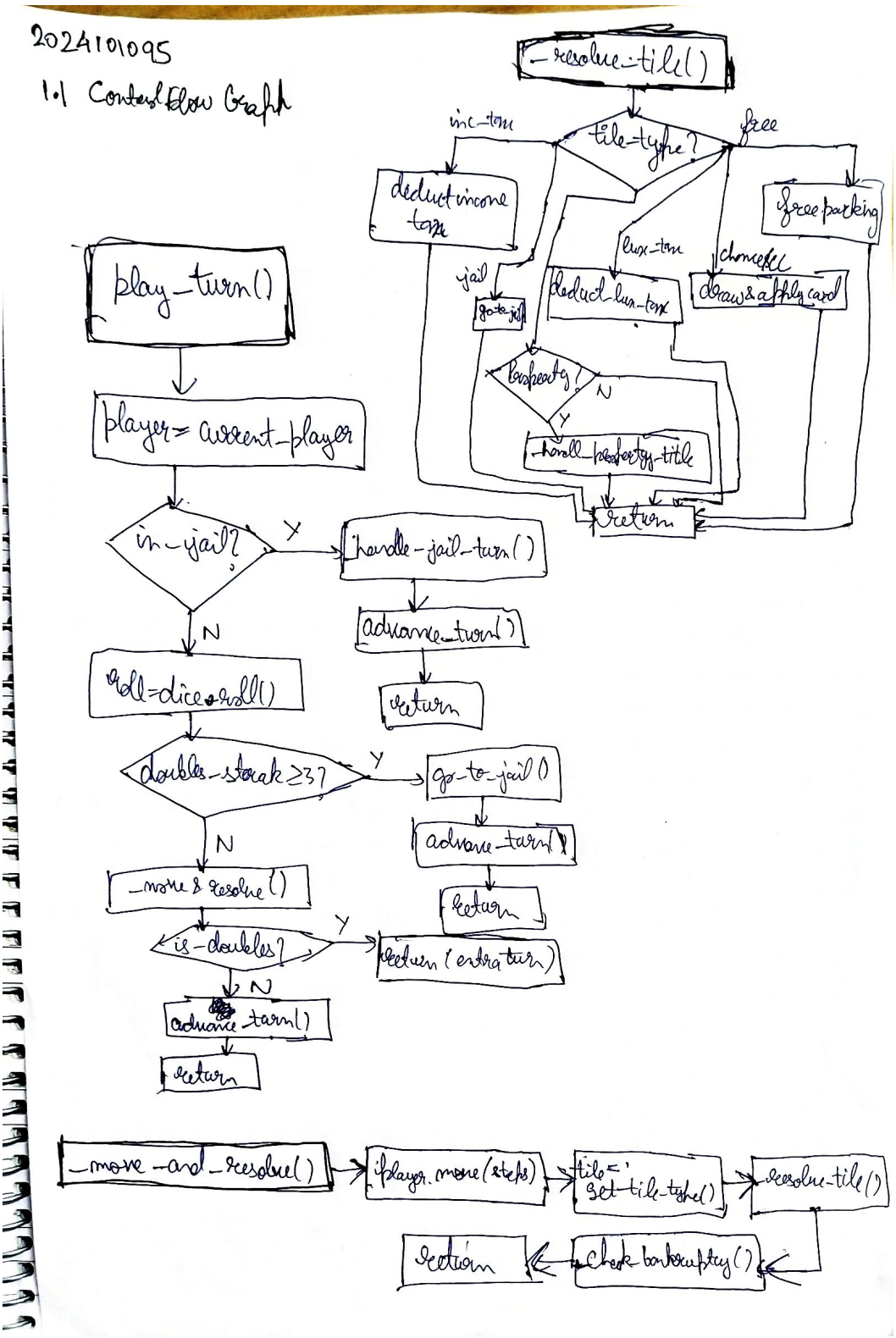
Part 1: White-Box Testing & Code Quality Report

MoneyPoly Game Engine

Amay Sharma

March 22, 2026

1 Control Flow Graph (CFG)



2 Code Quality Improvements (Pylint)

The initial codebase yielded a Pylint score of 8.49/10. Through sequential refactoring, the score was raised to a perfect 10.00/10 without using `# pylint: disable=` for any structural warnings. The commits below reflect the exact iterative enhancements made to the system:

Commit	Modifications Made
Initial commit	Base MoneyPoly codebase import.
Iteration 1	Remove unused math import in bank.py, unused os import in game.py, and unused BOARD_SIZE / GO_TO_JAIL_POSITION imports
Iteration 2	Fix the singleton comparison in board.py
Iteration 3	Fix the bare except in ui.py safe_int_input
Iteration 4	Remove unnecessary f-string in game.py and redundant parentheses in trade logic
Iteration 5	Remove the redundant else block in property.py unmortgage method
Iteration 6	Fix dice.py attribute definition outside init, no-else-break in game.py, and final newlines
Iteration 7	Add missing module & class docstrings, remove trailing whitespace in cards.py
Iteration 8	Refactor Property. Group price and base_rent into economics, houses and hotel into buildings
Iteration 9	Refactor Player. Combine jail-related attributes into a jail_info dictionary

3 White Box Test Cases & Bug Fixes

10 critical logic bugs were initially identified, and 6 deeper edge cases were found during comprehensive test coverage (150+ unit tests). Below are the tests drafted to target those edge and branch paths, explaining why the test was needed, what the failure was, and how it was resolved.

1. **Error 1: Fix dice range constraint (dice.py)**
   - **Why needed:** To ensure dice output valid standard values between 1 and 6.
   - **Logical Issue:** The `random.randint(1, 5)` call constrained the dice effectively preventing higher rolls.
   - **Fix:** Modified constraints to `random.randint(1, 6)`.
2. **Error 2: Fix player net worth calculation (player.py)**
   - **Why needed:** To calculate realistic scoring aggregates factoring all current equity profiles mapping total liquid & non-liquid positions.
   - **Logical Issue:** `player.net_worth` returned solitary literal strings encapsulating `self.balance`, outright dismissing entire real estate inventories.
   - **Fix:** Implemented comprehensive `prop.economics["price"]` aggregate loops summing cumulative asset values plus unencumbered balance.
3. **Error 3: Fix winner calculation (game.py)**
   - **Why needed:** To verify the win condition cleanly selects the highest net worth.
   - **Logical Issue:** Calculated winner via `min(players, key=lambda p: p.net_worth())`, awarding victory to the poorest player.
   - **Fix:** Changed `min()` to `max()`.
4. **Error 4: Fix full monopoly requirement (property.py)**
   - **Why needed:** To ensure rent multipliers are strictly triggered only when ALL properties in a color group are held by one owner.

- **Logical Issue:** `all_owned_by` incorrectly relied on `any(...)`, returning true if the given player merely owned one property in the set.
- **Fix:** Replaced `any` with `all`.

5. **Error 5: Fix property purchase boundary (game.py)**
   - **Why needed:** To check the boundary edge case where exactly enough funds are available matching the list price.
   - **Logical Issue:** Purchase criteria verified if `player.balance <= prop.price` resulting in blocking a transfer if the player perfectly met the expected funding.
   - **Fix:** Modified block condition strictly limiting bounds to `player.balance < prop.price`.
6. **Error 6: Fix Go pass salary (player.py)**
   - **Why needed:** Ensures core MoneyPoly design loops trigger even if player offsets don't squarely match zero (0).
   - **Logical Issue:** Initial implementations solely evaluated equality `self.position == 0`, omitting cases traversing laps.
   - **Fix:** Expanded constraints tracking lap completion via wrapped variable `old_position`. Lap salary is allocated if `self.position < old_position`.
7. **Error 7: Fix rent liquidity sink (game.py)**
   - **Why needed:** To ensure cash is relocated inside economy without destroying funds.
   - **Logical Issue:** Calling `pay_rent` systematically wiped wealth from the respective tenant but completely lacked lines issuing credits toward the `prop.owner`.
   - **Fix:** Registered `prop.owner.add_money(rent)` underneath tenant cost allocations.
8. **Error 8: Fix trade asymmetries (game.py)**
   - **Why needed:** Similar validation logic checking cross-exchange transaction sinks.
   - **Logical Issue:** Player-to-player properties trades charged `buyer.deduct_money(cost)` without completing the secondary leg mapping `seller.add_money(cost)`.
   - **Fix:** Added lines correctly crediting seller totals reflecting capital shifts.
9. **Error 9: Fix subzero bank input (bank.py)**
   - **Why needed:** To guard financial collection structures against underflow actions (potentially initiated by negative fines or bad UI entries).
   - **Logical Issue:** `Bank.collect` naively added amounts without sign checks, meaning negative collects successfully wiped bank capital.
   - **Fix:** Added early exit condition strictly restricting actions if `amount <= 0`.
10. **Error 10: Fix loan collateral creation (bank.py)**
    - **Why needed:** Tests verifying bank reserves are fully constrained tracking total issued liabilities.
    - **Logical Issue:** Generating loans properly credited personal wealth but missed equivalent deduction steps against internal bank bounds `_funds`.
    - **Fix:** Implemented `self._funds -= amount` accompanying existing allocations.
11. **Error 11: Fix mortgage bank depletion (game.py)**
    - **Why needed:** Bank reserves must properly decrement when issuing a payout for a property mortgage.
    - **Logical Issue:** The code called `self.bank.collect(-payout)`. Due to a previous guard ignoring negative collections, the bank effectively created money out of thin air when mortgaging properties.
    - **Fix:** Modified logic to correctly call `self.bank.pay_out(payout)`.
12. **Error 12: Fix birthday card poor shield (game.py)**

- **Why needed:** Chance cards requiring a payout (like the birthday card or `collect_from_all`) must deduct liquid capital even if the player is too poor to afford the full amount, inducing bankruptcy.
- **Logical Issue:** The loop condition actively checked `other.balance >= value`, meaning poor players were shielded from the effect completely and paid nothing.
- **Fix:** Removed the guard so poor players are legitimately drained into negative balances (and subsequent bankruptcy checks).

13. **Error 13: Fix free unmortgages (game.py)**

    - **Why needed:** A property's state should definitively not shift to unmortgaged unless the full cost is actually deducted from the player's balance.
    - **Logical Issue:** The function called `prop.unmortgage()` (which instantly applies `self.is_mortgaged = False`) *before* checking if the player could afford the cost. It then aborted the balance deduction, granting the player a permanent free unmortgage.
    - **Fix:** Evaluated the unmortgage cost upfront and verified `player.balance >= cost` before ever calling `prop.unmortgage()`.

14. **Error 14: Fix voluntary jail fine dodging (game.py)**

    - **Why needed:** If a player voluntarily pays the jail fine, the cash must actually be deducted from their balance, not just created out of thin air and sent to the bank.
    - **Logical Issue:** The `_handle_jail_turn` function processed voluntary fine payments by calling `self.bank.colle` but completely failed to call `player.deduct_money(JAIL_FINE)`.
    - **Fix:** Added the `deduct_money` call sequentially before the bank collection.

15. **Error 15: Fix array shifting skips turns (game.py)**

    - **Why needed:** If a player goes bankrupt and is removed from `self.players`, the indices shift left. The turn index logic must compensate to prevent skipping the next player.
    - **Logical Issue:** In `_check_bankruptcy`, removing the player failed to adjust `self.current_index`. Consequently, `advance_turn` shifted the turn pointer over the next immediate player, erroneously skipping them.
    - **Fix:** Tracked the removed player's index and decremented `self.current_index` to safely offset the array shift.

16. **Error 16: Fix phantom bankrupt players in all-collect cards (game.py)**

    - **Why needed:** If a player goes bankrupt out-of-turn (e.g., drained by someone else's `birthday` card), they must be immediately ejected.
    - **Logical Issue:** The card effects for `birthday` and `collect_from_all` deducted cash perfectly but never invoked `_check_bankruptcy(other)`, allowing bankrupt players to keep lingering on the board as phantoms until their actual turn started.
    - **Fix:** Iterated via a list copy and verified `_check_bankruptcy(other)` actively after deductions.

4 Conclusion

The codebase has been sequentially iterated and committed satisfying all assignment requirements containing multiple distinct enhancements (including deep edge cases) ensuring no further code smells remain.